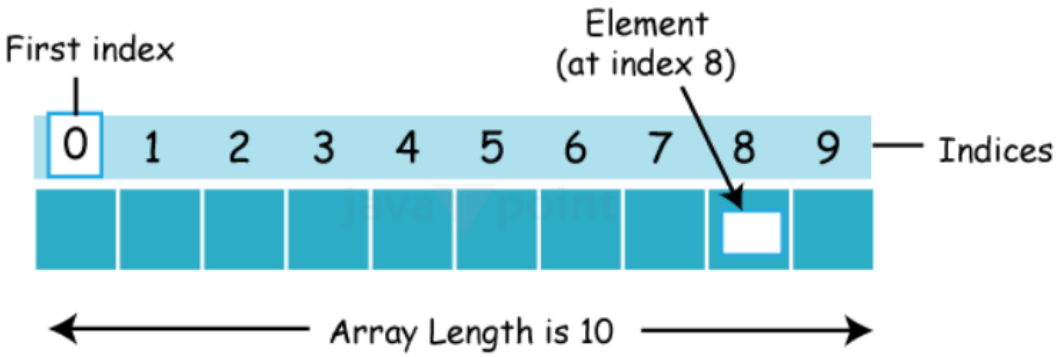
Java Arrays

Java arrays are objects that store multiple elements of the same data type in a single, contiguous memory block. In this chapter, we will learn about Java arrays, their types, features, and how to use them in programs.

What is an Array?

An array is a collection of elements of the same type stored in a single, contiguous block of memory. Arrays allow us to group similar data together and access them using an index.



Arrays are of two types:-

single dimensional and multi-dimensional.

- Single-Dimensional Array in Java
  - A single-dimensional array in Java is a linear collection of elements of the same data type. It can be declared and instantiated using the following syntax.

```
public class Main{
 public static void main(String args[]){
   //declaration and instantiation of an array
   int a[]=new int[5];
   a[0]=10;//initialization
   a[1]=20;
   a[2]=70;
   a[3]=40;
   a[4]=50;
   //traversing array
   for(int i=0;i<a.length;i++){//length is the property of array
      System.out.println(a[i]);
   }
 }
}
```

➢ Multidimensional Array (Two-Dimensional Array)

- A multidimensional array (two-dimensional array) in Java is an array of arrays, where each element of the main array is itself an array. It is used to store data in a **table-like structure** with rows and columns.

❖ Syntax of Declaring a Two-Dimensional Array

```
dataType[][] arrayName;
```

```java
public class Main {
    public static void main(String[] args) {
        // Declare and instantiate a 2D array
        int[][] numbers = new int[2][3];

        // Initialize the array
        numbers[0][0] = 10;
        numbers[0][1] = 20;
        numbers[0][2] = 30;
        numbers[1][0] = 40;
        numbers[1][1] = 50;
        numbers[1][2] = 60;

        // Traverse and print the array
        for (int i = 0; i < numbers.length; i++) {
            for (int j = 0; j < numbers[i].length; j++) {
                System.out.print(numbers[i][j] + " ");
            }
            System.out.println();
        }
    }
}
```

Multiplication Two Multidimensional Arrays (Two Matrices)

```
//Java Program to multiply two matrices
public class MatrixMultiplicationExample{
public static void main(String args[]){
//creating two matrices
int a[][]={{1,1,1},{2,2,2},{3,3,3}};
int b[][]={{1,1,1},{2,2,2},{3,3,3}};

//creating another matrix to store the multiplication of two matrices
int c[][]=new int[3][3];  //3 rows and 3 columns

//multiplying and printing multiplication of 2 matrices
for(int i=0;i<3;i++){
for(int j=0;j<3;j++){
c[i][j]=0;
for(int k=0;k<3;k++)
{
c[i][j]+=a[i][k]*b[k][j];
}//end of k loop
System.out.print(c[i][j]+" ");  //printing matrix element
}//end of j loop
System.out.println();//new line
}
}}
```

Control Statements in Java

Java compiler executes the code from top to bottom. The statements in the code are executed according to the order in which they appear.
However, Java provides statements that can be used to control the flow of Java code. Such statements are called control flow statements. It is one of the fundamental features of Java, which provides a smooth flow of program.

1. Decision Making statements
    - if statements
    - switch statement
2. Loop statements
    - do while loop
    - while loop
    - for loop
    - for-each loop

3. Jump statements
    - break statement
    - continue statement

1) If Statement

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition. The condition of the If statement gives a Boolean value, either true or false. In Java, there are four types of if-statements given below.

1. Simple if statement
2. if-else statement
3. if-else-if ladder
4. Nested if-statement

```
public class Main {
  public static void main(String[] args) {
    int x = 10;
    int y = 12;
    if(x+y > 20) {
      System.out.println("x + y is greater than 20");
    }
  }
}
```

1.2) if-else statement

The if-else statement is an extension to the if-statement, which uses another block of code, i.e., else block. The else block is executed if the condition of the if-block is evaluated as false.

```
public class Main {
  public static void main(String[] args) {
    int x = 10;
    int y = 12;
    if(x+y < 10) {
      System.out.println("x + y is less than 10");
    }  else {
      System.out.println("x + y is greater than 20");
    }
  }
}
```

1.3) if-else-if ladder

```
public class Main {
   public static void main(String[] args) {
      String city = "Delhi";
      if(city == "Meerut") {
      System.out.println("city is meerut");
      }else if (city == "Noida") {
      System.out.println("city is noida");
      }else if(city == "Agra") {
      System.out.println("city is agra");
      }else {
      System.out.println(city);
      }
   }
}
```

1.4) Nested if-statement

```
public class Main {
   public static void main(String[] args) {
      String address = "Delhi, India";

      if(address.endsWith("India")) {
         if(address.contains("Meerut")) {
         System.out.println("Your city is Meerut");
         }else if(address.contains("Noida")) {
         System.out.println("Your city is Noida");
         }else {
         System.out.println(address.split(",")[0]);
      }
      }else {
         System.out.println("You are not living in India");
      }
   }
}
```

2) Switch Statement

```
public class Main {
  public static void main(String[] args) {
    int num = 2;
    switch (num){
      case 0:
      System.out.println("number is 0");
      break;
      case 1:
      System.out.println("number is 1");
      break;
      default:
      System.out.println(num);
    }
  }
}
```

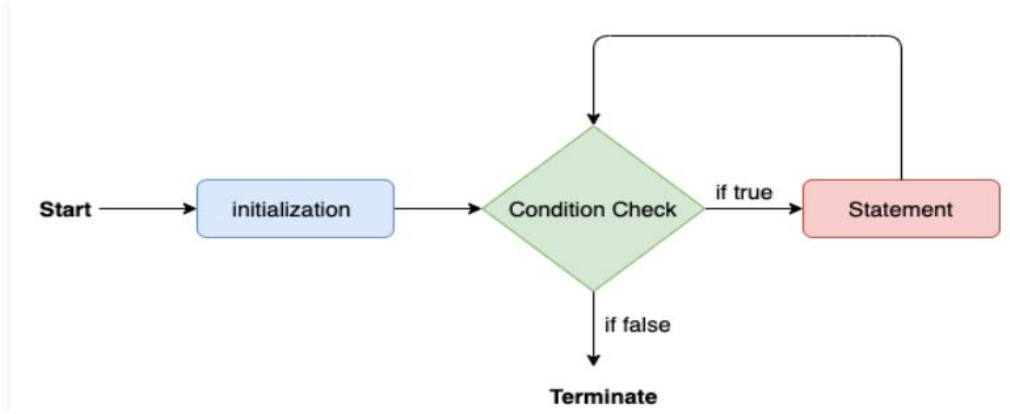
Java Loop Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of instructions in a repeated order. The execution of the set of instructions depends upon a particular condition.

1. Java for loop
2. Java for-each loop
3. Java while loop
4. Java do-while loop

Java for loop

In Java, for loop is similar to C and C++. It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.




```
public class Main {
   public static void main(String[] args) {
      // TODO Auto-generated method stub
      int sum = 0;
      for(int j = 1; j<=10; j++) {
         sum = sum + j;
      }
      System.out.println("The sum of first 10 natural numbers is " + sum);
   }
}
```

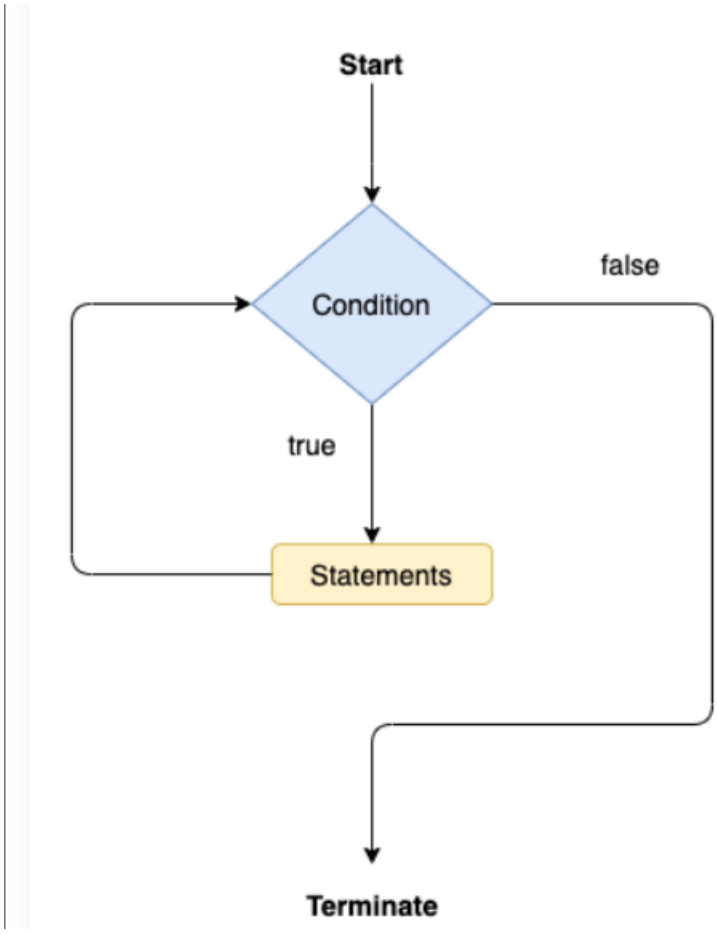
Java for-each loop

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable.

```
public class Main {
   public static void main(String[] args) {
      String[] names = {"Java","C","C++","Python","JavaScript"};
      System.out.println("Printing the content of the array names:");
      for(String name:names) {
         System.out.println(name);
      }
   }
}
```

Java while loop

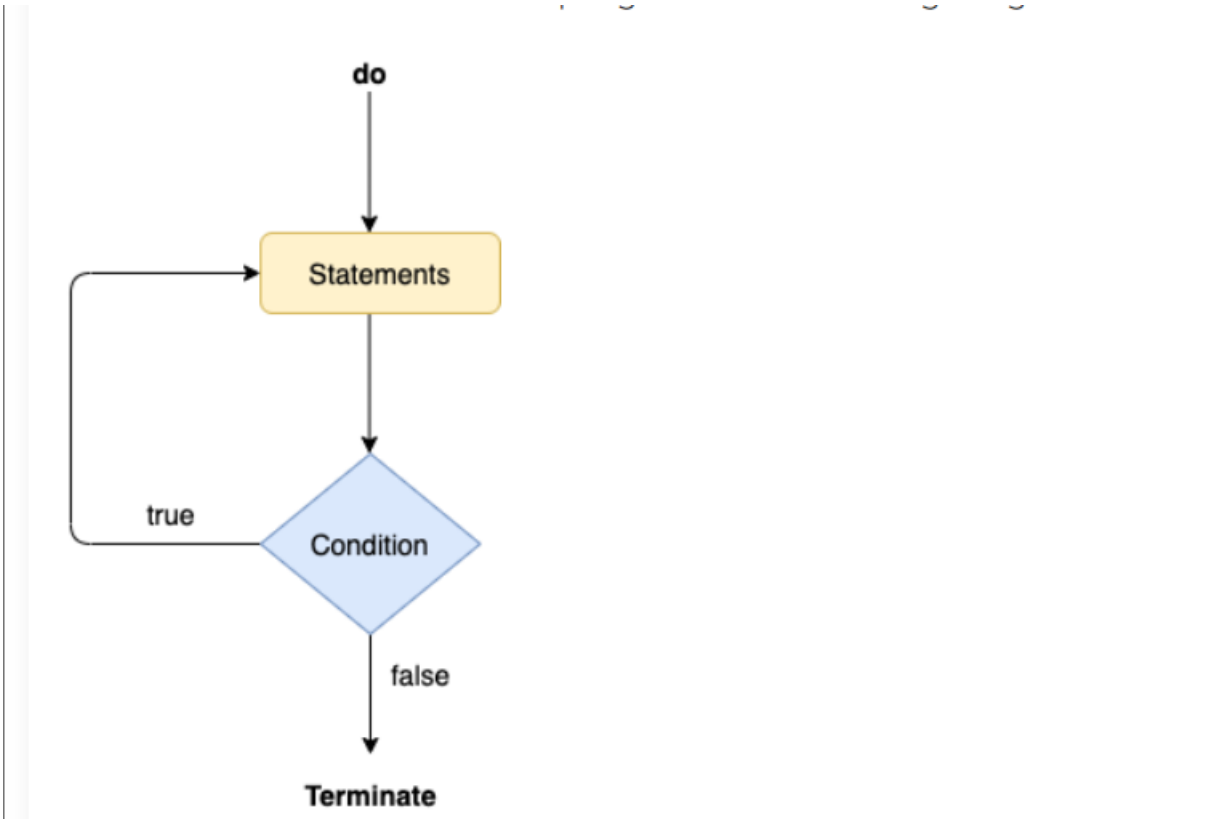
The while loop is also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop. Unlike for loop, the initialization and increment/decrement doesn't take place inside the loop statement in while loop.




```
public class Main {
  public static void main(String[] args) {
    int i = 0;
    System.out.println("Printing the list of first 10 even numbers");
    while(i<=10) {
      System.out.println(i);
      i = i + 2;
    }
  }
}
```

Java do-while loop

The do-while loop checks the condition at the end of the loop after executing the loop statements. When the number of iteration is not known and we have to execute the loop at least once, we can use do-while loop.




```
public class Main {
  public static void main(String[] args) {
    int i = 0;
    System.out.println("Printing the list of first 10 even numbers ");
    do {
      System.out.println(i);
      i = i + 2;
    }while(i<=10);
  }
}
```

Java Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program. There are two types of jump statements in Java. These are as listed below:

1. Java break statement
2. Java continue statement

Java break statement

As the name suggests, the break statement is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement. However, it breaks only the inner loop in the case of the nested loop.

```
public class Main {
  public static void main(String[] args) {
    for(int i = 0; i<= 10; i++) {
      System.out.println(i);
      if(i==6) {
      break;
      }
    }
  }
}
```

```
public class Main {
public static void main(String[] args) {
  a:
  for(int i = 0; i<= 10; i++) {
    b:
    for(int j = 0; j<=15;j++) {
      c:
      for (int k = 0; k<=20; k++) {
        System.out.println(k);
        if(k==5) {
        break a;
        }
      }
    }

  }
}
}
```

Java continue statement

Unlike break statement, the continue statement doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

```java
public class Main {
public static void main(String[] args) {
  for(int i = 0; i<= 2; i++) {

    for (int j = i; j<=5; j++) {

      if(j == 4) {
      continue;
      }
      System.out.println(j);
    }
  }
}
}
```

```java
StudentManagementSystem.java > StudentManagementSystem > main(String[])
import java.util.Scanner;

public class StudentManagementSystem {
    Run | Debug
    public static void main(String[] args) {
        int[] marks = new int[5];  // Array to store marks of 5 students
        Scanner scanner = new Scanner(System.in); // Create Scanner object
        int total = 0;   // Variable to store total marks

        System.out.println(x: "Student Marks Entry System");


        // Loop to take input for 5 students
        for (int i = 0; i < 5; i++) {
            System.out.print("Enter marks for Student " + (i + 1) + " (0-100): ");
            marks[i] = scanner.nextInt();
        }

        System.out.println(x: "\n Processing Results");

        int count = 1;  // Counter to display student number

        // Enhanced for loop to process each student's marks
        for (int mark : marks) {
            total += mark;

            String grade;   // Variable to store grade

            if (mark >= 90) {
                grade = "A+ (Excellent)";
            } else if (mark >= 75) {
                grade = "A (Very Good)";
            } else if (mark >= 50) {
                grade = "B (Pass)";
            } else {
                grade = "F (Fail)";
            }
```

```
StudentManagementSystem.java > StudentManagementSystem > main(String[])
public class StudentManagementSystem {
    public static void main(String[] args) {
            } else {
                grade = "F (Fail)";
            }

            System.out.println("Student " + count + ": Mark = " + mark + " | Grade = " + grade);
            count++;
        }
            // Calculate average marks

        double average = (double) total / marks.length;
        System.out.println(x: "--------------------------");
        System.out.println("Average Score: " + average);

        if (average >= 50) {
            System.out.println(x: "Final Result: CLASS PROMOTED");
        } else {
            System.out.println(x: "Final Result: CLASS REPEATED");
        }

        scanner.close();
    }
}
```

Name:-Ragav Sharma

PRN:-202401110012